

Introducción

Descripción de la base de datos:

Esta base de datos explora diferentes hábitos personales de la vida de los participantes, quienes juegan videojuegos como pasatiempo. Aparte de categorizar por edad, género y variedad de juego preferida, el estudio sintético documenta horas de sueño, calificaciones, horas de juego, dolor de cuello y vista, etc.

“Este conjunto de datos fue generado sintéticamente utilizando marcos clínicos establecidos e investigación revisada por pares para garantizar una representación realista y basada en evidencia de los patrones de comportamiento relacionados con los videojuegos. Las clasificaciones diagnósticas y de riesgo de adicción se modelaron de acuerdo con los criterios del Trastorno por Videojuegos del World Health Organization (ICD-11) y con investigaciones de la American Psychological Association sobre el Trastorno por Juego en Internet, incorporando indicadores psicológicos validados como cambios en el estado de ánimo, síntomas de abstinencia y deterioro funcional.

Las relaciones estadísticas entre la duración del juego y los resultados en salud se basaron en hallazgos publicados en el Journal of Behavioral Addictions, mientras que las variables de duración del sueño y alteraciones del ritmo circadiano se guiaron por estudios de la Sleep Research Society.

Las distribuciones demográficas, las tendencias de comportamiento de los jugadores y los patrones del mercado se sintetizaron utilizando informes de la industria de Newzoo y la Entertainment Software Association. En conjunto, estas fuentes orientaron un proceso de simulación estructurado que refleja patrones de riesgo clínicamente fundamentados, proporciones demográficas realistas y correlaciones conductuales respaldadas empíricamente.”

En mi análisis agrupe por estudiantes y trabajadores, al igual que aquellos que trabajan y estudian a la vez. Es así que filtré mis datos en 3 grupos distintos, y basé mi código en esto para poder generar funciones que faciliten el proceso de análisis.

Desarrollo

Lo primero que se encuentra en mi código es la importación de las siguientes bibliotecas: **pandas y matplotlib**. También se definen los *data frames* y sus diferentes grupos, para después desarrollar mis funciones.

Ya que mi base de datos tiene una gran cantidad de columnas distintas, desarrollé una función interactiva, la cual me permite intercambiar y escoger entre diferentes tipos de valores para producir gráficas. Estas utilizan *input()* y le presentan un menú al usuario.

```

while True:
    print("\n====Herramienta de Análisis de Datos en: 'Gaming and Mental Health'====")
    print("\n1. Producir una gráfica de barras")
    print("\n2. Comparar columnas entre grupos")
    print("\n3. Filtro por juego")
    print("\n4. Edad promedio participantes")
    print("\n5. Cantidad de participantes por grupo")
    print("\n6. Gráficas de dispersión sueño y ejercicio")
    print("\n7. Salir")

    opcion=input("\nSelecciona una opción del Menú: ")

    if opcion == "1":
        distribucion_columna_str()

    elif opcion == "2":
        barras_comparativas()

    elif opcion == "3":
        filtro_por_juego(df, )

    elif opcion == "4":
        promedio_edad(df)

    elif opcion == "5":
        poblacion()

    elif opcion == "6":

        plt.figure()
        plt.scatter(df["exercise_hours_weekly"], df["weight_change_kg"]) #tener la misma dimension de datos (x, y)
        plt.title("Relación horas de ejercicio vs peso en kilos adquirido")
        plt.xlabel("exercise_hours_weekly")
        plt.ylabel("weight_change_kg")
        plt.tight_layout()
        plt.show()

        plt.figure()
        plt.scatter(df["daily_gaming_hours"], df["sleep_hours"]) #tener la misma dimension de datos (x, y)
        plt.title("Relación horas de juego vs horas de sueño")
        plt.xlabel("daily_gaming_hours")
        plt.ylabel("sleep_hours")
        plt.tight_layout()
        plt.show()

    elif opcion == "7":
        print("Procesamiento terminado")
        break
    else:
        print("Opción no válida")

```

Filtración y agrupamiento de datos

El *dataset* cuenta con dos columnas: *gpa_score* y *work_performance*, en referencia a sus calificaciones (GPA, sistema americano) y cómo perciben que les va en su trabajo. Algunas filas tienen valores nulos en estas columnas, es de esta forma que podemos utilizar la función *notna()* de *pandas* para determinar campos vacíos.

```

import pandas as pd
import matplotlib.pyplot as plt
from pandas.api.types import is_numeric_dtype

df=pd.read_csv("C:\\Users\\Tati\\Documents\\Curso Python Tecmilenio\\Procesamiento de datos\\proyecto final\\gaming.csv", encoding="utf-8")

#Eliminación de nulos y agrupamiento para análisis
df_estudiantes = df[
    df["grades_gpa"].notna() &
    df["work_productivity_score"].isna()]

df_empleados = df[
    df["work_productivity_score"].notna() &
    df["grades_gpa"].isna()]

df_ambos = df[
    df["grades_gpa"].notna() &
    df["work_productivity_score"].notna()]

#Definición de columnas str
columnas_validas = df.select_dtypes(include=["object", "category", "bool"]).columns.tolist()

```

Código para generación de gráficas de barras

El menú inicia preguntándole al usuario que *data frame* desean usar entre los diferentes grupos disponibles, luego se pide la columna que desean analizar. Una vez que estos valores son definidos por el usuario se insertan en una generación de gráfica de barras con *matplotlib*.

```

#Creación de gráfica utilizando user input
def distribucion_columna_str():

    print("Tablas disponibles:")
    print("\n1. Empleados")
    print("\n2. Estudiantes")
    print("\n3. Trabajan y estudian")
    print("\n4. Todos")

    opcion_tabla = input("\nElige una opción (1-4): ")

    #creación de lista de posibles respuestas
    tablas = {
        "1": ("df_empleados", df_empleados),
        "2": ("df_estudiantes", df_estudiantes),
        "3": ("df_ambos", df_ambos),
        "4": ("df", df)
    }
    #tupla en caso de error
    if opcion_tabla not in tablas:
        print("Opción inválida, introduzca un número del 1 al 4: ")
        return

    #Aquí se define el df que se va a utilizar, y el nombre que se presentará en el título de la tabla

    nombre_df, df_usuario = tablas[opcion_tabla]

    #convertir una lista del diccionario a una lista hecha texto, esto es más "future proof" que la lista de arriba
    print("\nColumnas disponibles:")
    for i, col in enumerate(columnas_validas[1:], 1):
        print(f"{i}. {col}")

    columna_usuario = input("\nEscribe el nombre de la columna de la lista arriba: ")

    if columna_usuario not in columnas_validas:
        print("Columna inválida.")
        return

    #Se terminan de definir los parametros para la gráfica de barras utilizando las respuestas del usuario
    #se instruye contar valores y se organizan de menor a mayor

    frecuencia = df_usuario[columna_usuario].value_counts().sort_values()

    plt.figure()
    frecuencia.plot(kind="bar")

    plt.title(f"Frecuencia de {columna_usuario} en {nombre_df}")
    plt.xlabel(columna_usuario)
    plt.ylabel("Frecuencia")

    plt.tight_layout()
    plt.show()

    return frecuencia

```

El grupo "columnas_validas" fue definido junto con los *dataframes* del resto de la base, donde cualquier columna con valores *object*, *category* o *bool* se filtran a una lista separada. Esto asegura que se estará considerando una columna de la tabla que tiene valores *str* que se podrán aplicar a una gráfica de barras.

Código para generación de gráficas de barras comparativas

Aquí el concepto es muy similar al código anterior: Se busca facilitar el análisis de datos utilizando *input()*. Se utilizan desde un principio los 3 *data frames* disponibles para generar una gráfica de barras comparativa. Aquí se consideran dos posibilidades: Una en la que la columna contiene valores *string* y otra con valores numéricos.

```

#-----
#Barras juntas

def barras_comparativas():

    print("\nColumnas disponibles:")
    for i, col in enumerate(df[1:], 1):
        print(f"{i}. {col}")

    columna_comparar = input("\n¿Qué valor deseas comparar? ")

    if columna_comparar not in df.columns:
        print("La columna no existe. ")
        return

    #Para valores numéricos: Barra promedio
    if is_numeric_dtype(df_estudiantes[columna_comparar]):

        estudiantes = df_estudiantes[columna_comparar].mean()
        empleados = df_empleados[columna_comparar].mean()
        ambos = df_ambos[columna_comparar].mean()

        df_comparativa = pd.DataFrame({
            "Grupo": ["Estudiantes", "Empleados", "Trabajan y estudian"],
            "Promedio": [estudiantes, empleados, ambos] })

        ax = df_comparativa.plot(kind="bar", x="Grupo", y="Promedio", figsize=(8,5))
        plt.title(f"Comparación de {columna_comparar}")
        plt.xlabel("Grupo")
        plt.ylabel("Promedio")
        plt.legend()

        for barra in ax.patches:
            altura = barra.get_height()
            ax.text(
                barra.get_x() + barra.get_width()/2,
                altura,
                f"{altura:.2f}",
                ha="center",
                va="bottom"
            )

    plt.show()

```

Para los valores *string* se debe considerar cada una de las posibles respuestas del *dataset*, con valores numéricos sólo se tiene que generar el promedio de cada *data frame* con *mean()*.

```

plt.show()

#Para valores categóricos se crean barras comparativas
else:

    estudiantes = df_estudiantes[columna_comparar].value_counts()
    empleados = df_empleados[columna_comparar].value_counts()
    ambos = df_ambos[columna_comparar].value_counts()

    df_comparativa = pd.concat(
        [estudiantes, empleados, ambos],
        axis=1,
        keys=["Estudiantes", "Empleados", "Trabajan y estudian"]
    ).fillna(0)

    df_comparativa.plot(kind="bar", figsize=(8,5))
    plt.ylabel("Frecuencia")

    plt.title(f"Comparación de {columna_comparar}")
    plt.xlabel("Grupo / Categoría")
    plt.tight_layout()

    ax = df_comparativa.plot(kind="bar", figsize=(8,5))

    for barra in ax.patches:
        altura = barra.get_height()
        ax.text(
            barra.get_x() + barra.get_width()/2,
            altura,
            int(altura),
            ha="center",
            va="bottom"
        )

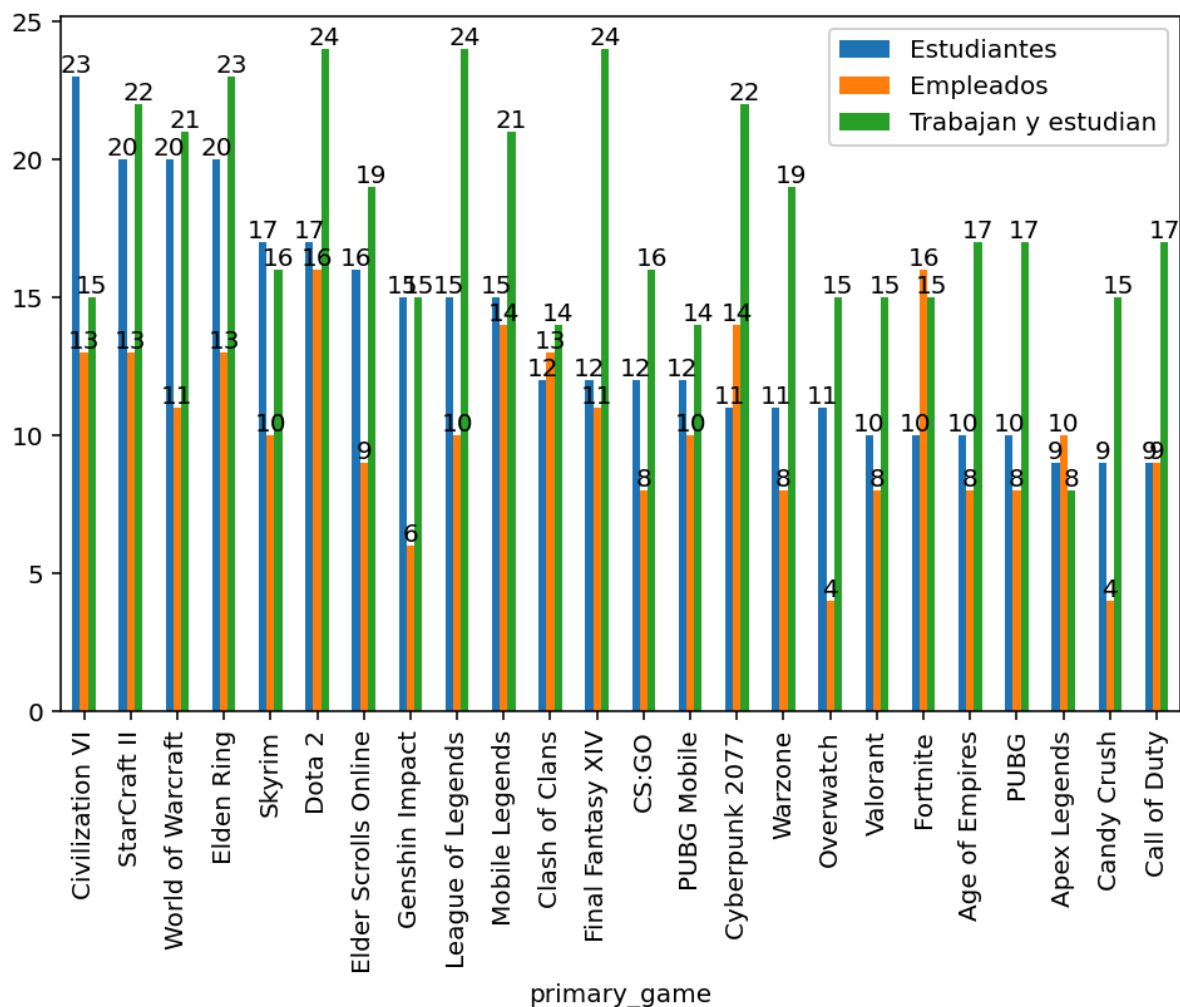
    plt.show()

#barras_comparativas()

```

Un límite de este formato es que en columnas como *primary_game* (el juego que más juegan los participantes) es que se están considerando demasiadas opciones, lo cual

resulta en gráficas muy difíciles de leer. Esto se podría corregir agregando una condición `if()` adicional que considere dimensiones más grandes para la gráficas, pero esto es más bien un problema estético.



Cálculo de poblaciones en *data frames*

La utilidad de esta función es para que el usuario tenga la cantidad de participantes categorizados en cada *data frame*, esta es una función separada ya que las otras opciones no incluyen `count()`, así que no hay forma directa de saber el número de participantes, estudiantes o trabajadores.

```
# Poblacion de participante en cada df

def poblacion():
    print("Número de participantes por grupo: ")
    print("\nEstudiantes:", len(df_estudiantes))
    print("Empleados:", len(df_empleados))
    print("Trabajan y estudian:", len(df_ambos))
    print("Todos:", len(df))
    return

#poblacion()
```

Filtro por juego

Utilizando la columna *primary_game* e *input()*, se pueden producir *data frames* que utilizan un juego específico. Al usuario se le entrega una lista de los juegos disponibles a escoger. El nuevo data frame se define, pero cualquier *query* necesaria es fuera del menú.

```
#funciones
def filtro_por_juego(df, ):

    print("\nJuegos disponibles:")
    juegos_disponibles = df["primary_game"].unique()
    for i, juego in enumerate(juegos_disponibles.tolist(), 1):
        print(f"{i}. {juego}")

    titulo = input("\nIntroduce el nombre de un juego: ")

    if titulo not in juegos_disponibles:
        print("Opción inválida. ")
        return

    df_filtro_por_juego = df[df['primary_game']==titulo]

    return print(df_filtro_por_juego)
```

Diagramas de dispersión

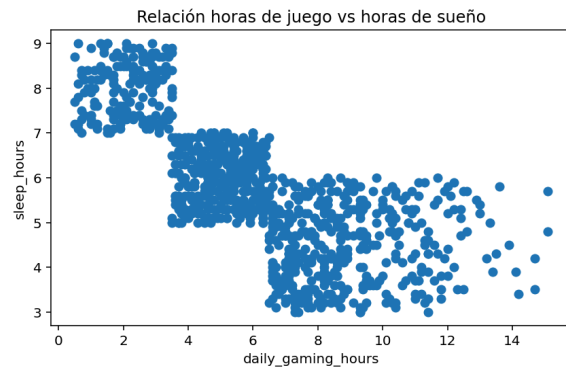
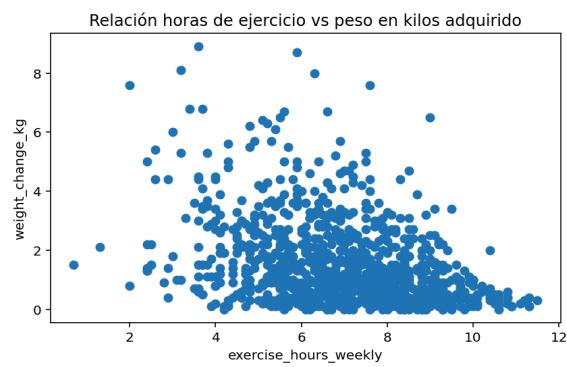
Aquí es donde se empieza a ver los límites de intentar hacer análisis con datos sintéticos, ya que en algunas gráficas se nota una tendencia pero en otras no se logra apreciar mucho.

```
def grafica_dispercion():

    plt.figure()
    plt.scatter(df["exercise_hours_weekly"], df["weight_change_kg"]) #tener la misma dimension de datos (x, y)
    plt.title("Relación horas de ejercicio vs peso en kilos adquirido")
    plt.xlabel("exercise_hours_weekly")
    plt.ylabel("weight_change_kg")
    plt.tight_layout()
    plt.show()

    plt.figure()
    plt.scatter(df["daily_gaming_hours"], df["sleep_hours"]) #tener la misma dimension de datos (x, y)
    plt.title("Relación horas de juego vs horas de sueño")
    plt.xlabel("daily_gaming_hours")
    plt.ylabel("sleep_hours")
    plt.tight_layout()
    plt.show()

    return
```



Implementación de Flask

```
(base) C:\Users\Tati\Documents\Curso Python Tecmilenio\Procesamiento de datos\proyecto final>python proyecto_final
python: can't open file 'C:\\Users\\Tati\\Documents\\Curso Python Tecmilenio\\Procesamiento de datos\\proyecto final\\proyecto_final': [Errno 2] No such file or directory

(base) C:\Users\Tati\Documents\Curso Python Tecmilenio\Procesamiento de datos\proyecto final>python proyecto_final.py

====Herramienta de Análisis de Datos en: 'Gaming and Mental Health'====

1. Producir una gráfica de barras
2. Comparar columnas entre grupos
3. Filtro por juego
4. Edad promedio participantes
5. Salir

Selecciona una opción del Menú: 5
Procesamiento terminado
* Serving Flask app 'proyecto_final'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
```

Formulario de Registro

Record ID:
1001

Age:
24

Gender:
Male

Daily Gaming Hours:
5.2

Game Genre:
MOBA

Primary Game:
Dota 2

Gaming Platform:
PC

Sleep Hours:
7

Sleep Quality:
Very Poor

Sleep Disruption Frequency:
Sometimes

Academic Work Performance:
Good

Grades GPA:

Work Productivity Score:

Mood State:
Irritable

Mood Swing Frequency:
Sometimes

Work Productivity Score:

Mood State:
Irritable

Mood Swing Frequency:
Sometimes

Withdrawal Symptoms:
False

Loss of Other Interests:
True

Continued Despite Problems:
True

Eye Strain:
True

Back Neck Pain:
True

Weight Change (kg):
4.5

Exercise Hours Weekly:
2.1

Social Isolation Score:
3

Face to Face Social Hours Weekly:
2.7

Monthly Game Spending USD:
21.56

Years Gaming:
14

Gaming Addiction Risk Level:
High

Guardar


```
File Edit Selection View Go Run Terminal Help
gaming.csv X
C:\Users\Tati\Documents\Curso Python Tecmilenio\Procesamiento de datos\proyecto final\gaming.csv
977 over,FALSE,FALSE,TRUE,TRUE,TRUE,0.8,4.8,7.7,6.59,84,2,Moderate
978 FALSE,FALSE,FALSE,TRUE,1.4,8.7,4.8,9,30.61,6,Low
979 E,TRUE,FALSE,TRUE,FALSE,1.5,7.4,5.4,8,113.01,6,High
980 JE,FALSE,1.7,5.3,6,3.8,44.54,9,High
981 lited,Sometimes,TRUE,TRUE,FALSE,TRUE,TRUE,3.1,6.9,5.2,4,107.16,12,Severe
982 RUE,FALSE,TRUE,TRUE,2.7,6.8,5.2,4,361.61,15,Severe
983 Irritable,Often,TRUE,TRUE,FALSE,TRUE,FALSE,1.4,2.4,8,0,438.83,2,Severe
984 ,Withdrawn,Never,FALSE,FALSE,FALSE,TRUE,FALSE,1.8,1.5,4,2,195.16,4,Low
985 ,TRUE,1.1,8.3,5,6.3,124.93,1,Low
986 ,FALSE,FALSE,FALSE,FALSE,FALSE,0.9,8.5,4,13,20.1,8,Low
987 RUE,FALSE,FALSE,FALSE,1.6,6.8,3,4.5,67.44,7,Moderate
988 FALSE,FALSE,FALSE,FALSE,0.1,6.9,1,8.5,1.15,6,Low
989 FALSE,FALSE,FALSE,FALSE,2.9,6.5,2,9,59.91,9,Low
990 times,FALSE,FALSE,FALSE,FALSE,FALSE,1.9,7.3,12.1,8.08,7,Low
991 FALSE,FALSE,FALSE,FALSE,0.1,9,1,12.2,45.29,2,Low
992 SE,TRUE,1.4,7.7,5,5.8,121.06,2,Moderate
993 ic,Rarely,FALSE,FALSE,FALSE,FALSE,FALSE,1.5,7.7,1,9.5,31.45,1,Low
994 Often,FALSE,FALSE,FALSE,TRUE,FALSE,1.8,8.3,3,8.2,143.48,8,Low
995 LSE,FALSE,FALSE,1.4,6.4,5,9,618.04,19,Low
996 pr,FALSE,TRUE,TRUE,FALSE,FALSE,0.4,4.4,6.6,1,220.04,10,Severe
997 ometimes,TRUE,FALSE,TRUE,TRUE,TRUE,1.9,7.5,6,2,426.54,3,Severe
998 ry,Often,FALSE,FALSE,FALSE,TRUE,FALSE,2.1,7.7,1,10.9,83.71,7,Low
999 FALSE,TRUE,0.5,8.1,5,6.7,88.6,5,High
1000 FALSE,FALSE,FALSE,FALSE,FALSE,0.8,8.4,1,12.7,22.02,8,Low
1001 al,Daily,FALSE,FALSE,FALSE,FALSE,FALSE,0.9,3.2,10.9,25.2,19,Low1001,24,Male,5.2,MOBA,Dota 2,PC,7,Very Poor,Sometimes,Good,,Irritable,Sometimes,False,True,True,True,True,4.5,2.1,3,2,7,21.56,14,High
1002
```

```
C:\Users\Tati\Documents\Curso Python Tecmilenio\Procesamiento de datos\proyecto final\proyecto_final.py
Actividades funciones.py x Actividades pandas y matplotlib.py x manipulacion de dataframes.py x proyecto_final.py x
270
271 ## USO DE FLASK
272
273
274 from flask import Flask, render_template, request, redirect
275 import csv
276 import os
277
278 app = Flask(__name__)
279
280 @app.route('/')
281 def formulario():
282     return render_template('formato.html')
283
284
285
286 @app.route('/guardar', methods=['POST'])
287 def guardar():
288     record_id = request.form['record_id']
289     age = request.form['age']
290     gender = request.form['gender']
291     daily_gaming_hours = request.form['daily_gaming_hours']
292     game_genre = request.form['game_genre']
293     primary_game = request.form['primary_game']
294     gaming_platform = request.form['gaming_platform']
295     sleep_hours = request.form['sleep_hours']
296     sleep_quality = request.form['sleep_quality']
297     sleep_disruption_frequency = request.form['sleep_disruption_frequency']
298     academic_work_performance = request.form['academic_work_performance']
299     grades_gpa = request.form['grades_gpa']
300     work_productivity_score = request.form['work_productivity_score']
301     mood_state = request.form['mood_state']
302     mood_swing_frequency = request.form['mood_swing_frequency']
303     withdrawal_symptoms = request.form['withdrawal_symptoms']
304     loss_of_other_interests = request.form['loss_of_other_interests']
305     continued_despite_problems = request.form['continued_despite_problems']
306     eye_strain = request.form['eye_strain']
307     back_neck_pain = request.form['back_neck_pain']
308     weight_change_kg = request.form['weight_change_kg']
309     exercise_hours_weekly = request.form['exercise_hours_weekly']
310     social_isolation_score = request.form['social_isolation_score']
311     face_to_face_social_hours_weekly = request.form['face_to_face_social_hours_weekly']
312     monthly_game_spending_usd = request.form['monthly_game_spending_usd']
313     years_gaming = request.form['years_gaming']
314     gaming_addiction_risk_level = request.form['gaming_addiction_risk_level']
315
```

```
C:\Users\Tati\Documents\Curso Python Tecnilenio\Proyecto de datos\proyecto final\proyecto_final.py

Actividades funciones.py x Actividades pandas y matplotlib.py x manipulacion de dataframes.py x proyecto_final.py x

312     monthly_game_spending_usd = request.form['monthly_game_spending_usd']
313     years_gaming = request.form['years_gaming']
314     gaming_addiction_risk_level = request.form['gaming_addiction_risk_level']
315
316
317     archivo_csv = 'gaming.csv'
318
319     campos = [
320         'record_id', 'age', 'gender', 'daily_gaming_hours', 'game_genre',
321         'primary_game', 'gaming_platform', 'sleep_hours', 'sleep_quality',
322         'sleep_disruption_frequency', 'academic_work_performance', 'grades_gpa',
323         'work_productivity_score', 'mood_state', 'mood_swing_frequency',
324         'withdrawal_symptoms', 'loss_of_other_interests',
325         'continued_despite_problems', 'eye_strain', 'back_neck_pain',
326         'weight_change_kg', 'exercise_hours_weekly', 'social_isolation_score',
327         'face_to_face_social_hours_weekly', 'monthly_game_spending_usd',
328         'years_gaming', 'gaming_addiction_risk_level'
329     ]
330
331     escribir_cabecera = (not os.path.exists(archivo_csv)) or (os.path.getsize(archivo_csv) == 0)
332
333     with open(archivo_csv, mode='a', newline='', encoding='utf-8') as f:
334         w = csv.DictWriter(f, fieldnames=campos)
```

```
C:\Users\Tati\Documents\Curso Python Tecnilenio\Proyecto de datos\proyecto final\proyecto_final.py

Actividades funciones.py x Actividades pandas y matplotlib.py x manipulacion de dataframes.py x proyecto_final.py x

330
331     escribir_cabecera = (not os.path.exists(archivo_csv)) or (os.path.getsize(archivo_csv) == 0)
332
333     with open(archivo_csv, mode='a', newline='', encoding='utf-8') as f:
334         w = csv.DictWriter(f, fieldnames=campos)
335
336         if escribir_cabecera:
337             w.writeheader()
338
339         w.writerow({
340             'record_id': record_id,
341             'age': age,
342             'gender': gender,
343             'daily_gaming_hours': daily_gaming_hours,
344             'game_genre': game_genre,
345             'primary_game': primary_game,
346             'gaming_platform': gaming_platform,
347             'sleep_hours': sleep_hours,
348             'sleep_quality': sleep_quality,
349             'sleep_disruption_frequency': sleep_disruption_frequency,
350             'academic_work_performance': academic_work_performance,
351             'grades_gpa': grades_gpa,
352             'work_productivity_score': work_productivity_score,
353             'mood_state': mood_state,
354             'mood_swing_frequency': mood_swing_frequency,
355             'withdrawal_symptoms': withdrawal_symptoms,
356             'loss_of_other_interests': loss_of_other_interests,
357             'continued_despite_problems': continued_despite_problems,
358             'eye_strain': eye_strain,
359             'back_neck_pain': back_neck_pain,
360             'weight_change_kg': weight_change_kg,
361             'exercise_hours_weekly': exercise_hours_weekly,
362             'social_isolation_score': social_isolation_score,
363             'face_to_face_social_hours_weekly': face_to_face_social_hours_weekly,
364             'monthly_game_spending_usd': monthly_game_spending_usd,
365             'years_gaming': years_gaming,
366             'gaming_addiction_risk_level': gaming_addiction_risk_level
367         })
368
369     return redirect('/')
370
371
372
373 if __name__ == '__main__':
374     app.run(debug=True, host='127.0.0.1', port=5000)
```

Conclusión

Esta información nos ayuda a comprender los efectos en la salud en diferentes estilos de vida y algunas de las preferencias de estas personas. Nos ayuda a entender a las personas que juegan videojuegos más a fondo y podría llegar a ser útil para equipos de mercadotecnia o para profesionales en el sector de salud mental.

Con las funciones en el código y el menú será más fácil manipular esta información para análisis más eficazmente. Aunque no se cubren todas las posibilidades, se tiene una mejor comprensión de los grupos segmentados.

Sin embargo, ya que esta base de datos está compuesta por información sintética, no se logra apreciar ningún tipo de patrón en los datos, incluso en algunos donde se esperaría una diferencia garantizada, como una diferencia en horas de sueño u horas de juego al día entre los grupos. Tampoco se nota algún tipo de correlación en ninguna de las gráficas de dispersión creadas.

Es posible que este *data set* se pobló a través de un comando de IA, ya que estos tienden a crear datos distribuidos equitativamente, lo cual resulta en la distribución de *primary_game* siendo igual cuando se consideran todos los participantes.

Las funciones diseñadas podrán encontrar un uso con otro tipo de *data set* o con una base de datos orgánica. La ventaja aquí sería una mejor comprensión de qué estilos de vida resultan en qué problemas y más allá de eso que segmentos del mercado padecen de estos estilos de vida.

La implementación de Flask no asiste mucho a la base de datos ya que los resultados vienen de un supuesto estudio en vez de una encuesta, pero parece funcionar y ajustarse a valores nulos.

Mi conclusión es que los *data_sets* sintéticos tienen su uso en practicar y modelar funciones y gráficas y son una buena herramienta para aprender de programas como Python y sus librerías, pero no se prestan para un análisis profundo.